Summary of chapter 1

Language models ==> Large Language models ==> Fundation models

A Language model encordes statistical information about one or more languages, it is able to predict what word is most likely to come after a sequence of words or before.

Masked Language Model

Masked language model (like BERT): predicts missing tokens anywhere in a sequence using the context on both sides, before and after the gap. It's good for understanding tasks such as classification, sentiment and code debugging. e.g "My favorite ___ is blue"

Auto regressive language model

Autoregressive language model (like GPT or Claude): predicts only the next token, using the tokens before it. This is the type that generates text, so it's the type most foundation models today use. e.g My favorite color is ____

Self-supervision:

what makes the step up to LLMs possible. The labels come from the text itself, so models can train on huge datasets without anyone labeling them by hand.

A token is the basic unit of a Language model, it is approximately 75% of a word. Tokenization is the basis of breaking text into tokens.

Advantages of tokens:

1. Tokens allow models to break words into meaningful components
2. There are fewer tokens than words so this makes the model vocabulary size smaller
3. Tokens allow the model to process unknown words

Example: BERT as a masked language model

BERT sees the words on **both sides** of `[MASK]` and guesses what fits in the gap.

Setup: the examples need `transformers` and `torch`. See "Running the notebooks" in the README.

In [4]:
```python
# "import" brings in code someone else wrote so we can use it.
# pipeline is a shortcut that bundles a model + everything needed to use
it.
from transformers import pipeline

# Create a "fill in the blank" tool using BERT.
#   "fill-mask"          -> the task: guess the word hidden by [MASK]
#   "bert-base-uncased"  -> which model to use ("uncased" = ignores capital
letters)
# The first time, this downloads the model (~440 MB). After that it loads
from disk.
unmasker = pipeline("fill-mask", model="bert-base-uncased")

# Our test sentence. [MASK] marks the word we want BERT to guess.
sentence = "The doctor wrote a [MASK] for the patient."

# Ask BERT for its top 5 guesses. It gives back a list, one item per guess.
# The "for" loop goes through that list one guess at a time.
for guess in unmasker(sentence, top_k=5):
    # Each guess is a dictionary (a set of labelled values). We use two of
them:
    #   guess['token_str'] -> the word BERT guessed
    #   guess['score']     -> how confident BERT is (0 to 1; 1 = 100% sure)
    # The f"..." string lets us put values inside text using {curly
braces}:
    #   :<15  -> pad the word to 15 characters so the columns line up
    #   :.3f  -> show the score with 3 decimal places
    print(f"{guess['token_str']:<15} {guess['score']:.3f}")
```

```
/Users/robertkigobe/Documents/My_Research/jira_rag_assistant/.venv/lib/pytho
n3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Pleas
e update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/st
able/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
W1026 19:18:07.112000 70323 torch/distributed/elastic/multiprocessing/redire
cts.py:35] NOTE: Redirects are currently not supported in MacOs.
Loading weights: 100%|██████████| 202/202 [00:00<00:00, 17598.97it/s]
[transformers] BertForMaskedLM LOAD REPORT from: bert-base-uncased
Key                          | Status     |  |
-----------------------------+------------+--+-
cls.seq_relationship.weight | UNEXPECTED |  |
cls.seq_relationship.bias   | UNEXPECTED |  |
bert.pooler.dense.bias      | UNEXPECTED |  |
bert.pooler.dense.weight    | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architectur
e; not ok if you expect identical arch.
letter          0.158
report          0.142
note            0.106
prescription    0.070
statement       0.033
```

Why context on both sides matters: change only the words *after* the mask and BERT's guesses change too. An autoregressive model (GPT) reading left to right would only see `"I went to the"` and could not use the rest.

In [7]:
```
# Two sentences that are identical BEFORE the [MASK] but different AFTER
it.
# If BERT gives different guesses, it proves it reads the words after the
gap too.
# The square brackets [ ] make a list; the loop takes one sentence at a
time.
for sentence in [
    "I went to the [MASK] to eat some bread.",
    "I went to the [MASK] to watch a movie.",
]:
    # Reuse the unmasker from the cell above (no need to load BERT again).
    # This line is a "list comprehension": a short way to build a list.
    # In plain words: "for each guess g, keep only its word (token_str)".
    top = [g["token_str"] for g in unmasker(sentence, top_k=3)]

    # Print the sentence (padded to 45 characters) followed by the 3
guesses.
    print(f"{sentence:<45} -> {top}")
```

```
I went to the [MASK] to eat some bread.       -> ['kitchen', 'market', 'bath
room']
I went to the [MASK] to watch a movie.        -> ['bathroom', 'theater', 'ci
nema']
```

Example of Autoregressive language model

In [8]:
```
from transformers import pipeline

# Create a text generator using GPT-2, a small autoregressive model (~500
MB).
# "text-generation" -> the task: continue the text we give it.
generator = pipeline("text-generation", model="gpt2")

# Give GPT-2 the start of a sentence and let it continue.
#   max_new_tokens=10 -> add at most 10 new tokens (roughly 7-8 words)
#   do_sample=False   -> always pick the single most likely next token,
#                        so you get the same answer every time you run it
out = generator("I went to the", max_new_tokens=10, do_sample=False)

# The result is a list holding one dictionary.
#   out[0]                    -> the first (and only) result in the list
#   ["generated_text"]        -> the full text: your prompt + what GPT-2
added
print(out[0]["generated_text"])
```

```
Loading weights: 100%|██████████| 148/148 [00:00<00:00, 5693.40it/s]
I went to the hospital and was told that I was going to be
```

In [10]:
```
# This cell does the same thing as the one above, but step by step,
# so you can watch GPT-2 add ONE token at a time (that's "autoregressive").
```

```python
# AutoTokenizer        -> turns text into token numbers (and back again)
# AutoModelForCausalLM -> loads a model that predicts the next token
#                         ("causal LM" is another name for autoregressive)
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch  # PyTorch: we use it to join lists of numbers (tensors) together

# Load GPT-2's tokenizer and the model itself.
tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")

# Turn our starting text into token numbers.
#   return_tensors="pt" -> give the numbers back in PyTorch format
#   .input_ids          -> the token numbers themselves
# Try: print(ids) to see that "My name" becomes a couple of numbers.
ids = tok("My name", return_tensors="pt").input_ids

# Repeat 6 times, adding one new token each time.
# The _ means "I don't need the loop counter, just repeat".
for _ in range(6):
    # 1. Ask the model to score EVERY word in its vocabulary (~50,000)
    #    as a possible next token. "logits" are those raw scores.
    #    [0, -1] means: first sentence, last position -> the "what comes next" scores.
    logits = model(ids).logits[0, -1]

    # 2. Pick the token with the highest score (argmax = "position of the biggest").
    #    .unsqueeze(0) just reshapes the number so it can be joined on below.
    next_id = logits.argmax().unsqueeze(0)

    # 3. Stick the new token onto the end of our sequence.
    #    Next time round the loop, the model sees this longer sequence.
    ids = torch.cat([ids, next_id.unsqueeze(0)], dim=1)

    # 4. Turn the numbers back into text and print it,
    #    so you see the sentence grow by one token each line.
    print(tok.decode(ids[0]))
```

```
Loading weights: 100%|██████████| 148/148 [00:00<00:00, 15576.95it/s]
My name is
My name is John
My name is John.
My name is John. I
My name is John. I'm
My name is John. I'm a
```

Supervision vs self-supervision

Supervised: humans label each example by hand (expensive, slow).

Self-supervised: the labels come from the data itself, e.g. the next word in a sentence (free, scales to the whole internet). This is how GPT-style models are trained.

In [11]:
```python
# SUPERVISED learning: a HUMAN had to write the label for every example.
# Here someone read each review and labelled it "positive" or "negative".
# Each item is a pair in round brackets ( ): (input, label)
labelled_reviews = [
    ("I loved this film",        "positive"),
    ("Terrible acting, so boring", "negative"),
    ("Best meal I've ever had",  "positive"),
    ("The service was awful",    "negative"),
]

# Show each example: the model would learn  input -> label
# "for text, label in ..." unpacks each pair into two variables.
for text, label in labelled_reviews:
    print(f"{text:<28} -> {label}")

# The problem: labelling millions of examples by hand is slow and
expensive.
```

```
I loved this film            -> positive
Terrible acting, so boring   -> negative
Best meal I've ever had      -> positive
The service was awful        -> negative
```

In [12]:
```python
# SELF-SUPERVISED learning: NO human labels needed.
# We take ordinary text and let the text itself provide the answers.
sentence = "I love street food in Kampala"

# Split the sentence into words (real models use tokens, but words are
easier to read).
words = sentence.split()

# For each position, the input is "the words so far" and the label is "the
next word".
# range(1, len(words)) counts 1, 2, 3, ... up to the number of words minus
1.
for i in range(1, len(words)):
    context = " ".join(words[:i])   # words[:i] -> the first i words,
joined back into text
    next_word = words[i]            # the word that comes right after ->
this is the label
    print(f"{context:<25} -> {next_word}")

# One sentence just gave us 5 training examples for free!
# Do this over the whole internet and you get trillions of examples,
# which is what makes training LARGE language models possible.
```

```
I                         -> love
I love                    -> street
I love street             -> food
I love street food        -> in
I love street food in     -> Kampala
```

Large Language models to Foundation models

From language models to LARGE language models

A language model becomes a **large** language model when two things grow together:

1. **Parameters**: the numbers the model learns during training. More parameters give it more capacity to learn patterns in language.
2. **Training data**: a bigger model needs more data to learn from, otherwise its extra capacity is wasted.

Self-supervision is the bridge. With supervised learning, the amount of data is limited by how fast humans can label it. With self-supervision, any text works as training data (books, websites, code), so data can grow to trillions of tokens. That much data is what makes very large models worth building.

How "large" has grown

Model	Year	Parameters
GPT-1	2018	117 million
BERT-base (my masked example)	2018	110 million
GPT-2 (my autoregressive example uses the small 124M version)	2019	1.5 billion
GPT-3	2020	175 billion

"Large" is relative: what counts as large today will look small in a few years.

Why size matters in practice

- **Memory ≈ parameters × bytes per parameter.** BERT-base at 4 bytes per parameter is about 440 MB; a 7B model is about 28 GB (quantization shrinks this).
- Larger models are better at following instructions and handling new tasks, but they cost more to train and run.

Summary: Language model + self-supervision → lots of data → more parameters → Large Language Model

From large language models to FOUNDATION models

LLMs only work with text. But people learn from more than text (we see, hear and read), so AI models expanded to handle other kinds of data too.

1. More than text: multimodal models

A **modality** is a type of data: text, images, audio, video.

- A model that works with more than one modality is **multimodal**.
- A large model that does this is a **Large Multimodal Model (LMM)**. Examples: GPT-4V, Gemini, and Claude (which can read images).

Self-supervision still applies. For example, OpenAI's CLIP (2021) learned from about 400 million (image, caption) pairs found on the internet. Nobody labelled them by hand; the caption written next to an image serves as its label.

2. Why "foundation"?

These models are called **foundation models** because:

- they are the **base** that many different applications are built on, and
- one general model can handle many tasks that used to need separate, specially trained models (translation, sentiment, summarising, describing images...).

Before, there was one model per task. Now there is one foundation model, adapted to many tasks.

3. Adapting a foundation model to your task

You rarely train a foundation model yourself. Instead you adapt one that already exists:

- **Prompt engineering**: give clear instructions and examples in the input.
- **RAG (retrieval-augmented generation)**: give the model relevant documents to use when answering.
- **Finetuning**: train it a little more on your own data.

Building applications this way is what the book calls **AI engineering**.

Summary: Language model → (self-supervision + scale) → Large Language Model → (more modalities + general purpose) → Foundation Model → (adapted by us) → AI applications

From foundation models to AI ENGINEERING

AI engineering is the process of building applications on top of foundation models that already exist. Instead of training a model from scratch, you take a ready-made model (often through an API, "model as a service") and adapt it to your problem.

Why AI engineering took off

1. **General-purpose capabilities**: one foundation model can do many tasks, so more applications become possible.

2. **More investment**: companies are spending heavily on AI, so models keep getting better and cheaper to use.
3. **Low barrier to entry**: you can build something useful with a few lines of code and an API key. No need for your own data, GPUs or ML expertise to get started.

AI engineering vs traditional ML engineering

	ML engineering	AI engineering
Starting point	Train your own model	Use an existing foundation model
Main work	Collecting data, training models	**Adapting** models (prompts, RAG, finetuning)
Model size	Usually small	Large, so more compute and slower responses (latency)
Outputs	Closed-ended (e.g. "positive"/"negative")	Open-ended (any text), so **evaluation is harder**

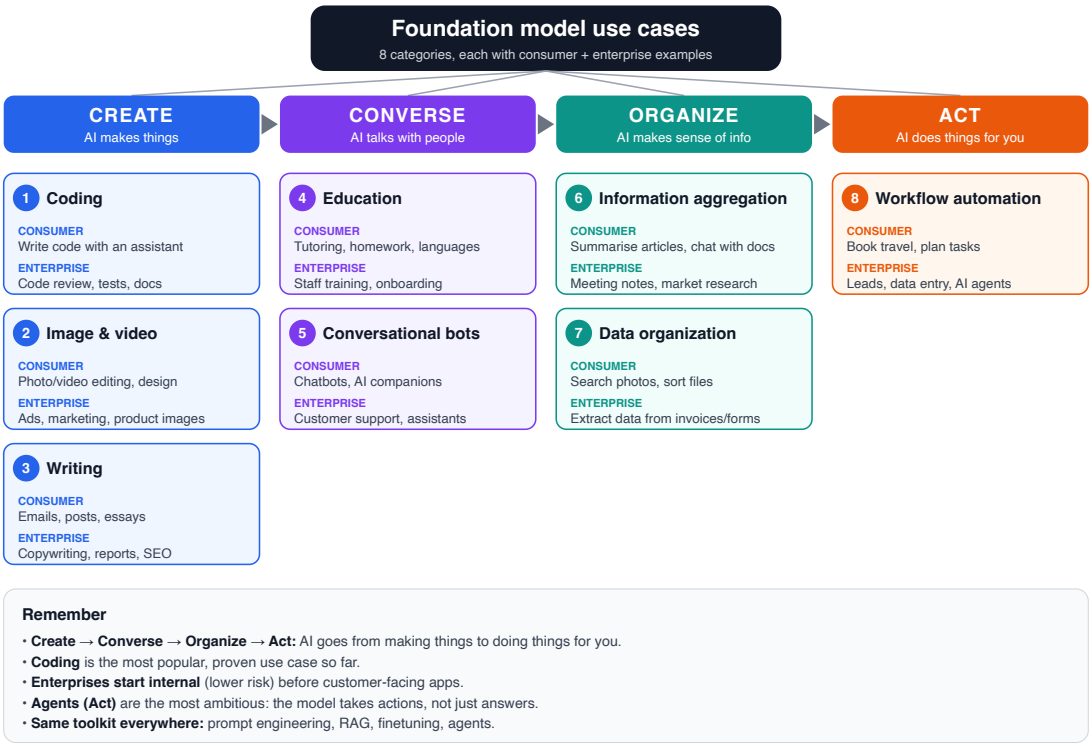
The AI engineering stack (three layers)

1. **Application development**: prompts, context, user interface, evaluation. Most AI engineers work here.
2. **Model development**: training, finetuning, datasets, inference optimisation (e.g. quantization).
3. **Infrastructure**: serving models, managing compute (GPUs), monitoring.

Full chapter summary: Language model → (self-supervision + scale) → Large Language Model → (more modalities + general purpose) → Foundation Model → (adapted to real problems) → **AI Engineering**

Foundation model use cases

The book groups use cases into **8 categories**, each with consumer and enterprise examples.



Memory aid (my own grouping, not the book's): **Create → Converse → Organize → Act**. AI goes from making things, to talking with people, to making sense of information, to taking actions for you.

Category	Consumer examples	Enterprise examples
1. Coding	Writing code with an AI assistant	Code review, generating tests, documentation
2. Image & video production	Photo/video editing, design	Ads, marketing content, product images
3. Writing	Emails, social media posts, essays	Copywriting, reports, SEO content
4. Education	Tutoring, homework help, language learning	Employee training and onboarding
5. Conversational bots	General chatbots, AI companions	Customer support, product assistants
6. Information aggregation	Summarising articles, "talk to your docs"	Summarising meetings, market research
7. Data organization	Searching photos, organising files	Extracting data from documents (invoices, forms), knowledge management
8. Workflow automation	Booking travel, planning, everyday tasks	Processing leads, data entry, **AI agents** that use tools

Key points

- **Coding** is one of the most popular and proven use cases so far.
- **Enterprises usually start with internal-facing, lower-risk uses** (e.g. a tool for employees to search internal documents) before customer-facing ones, because mistakes inside the company are cheaper than mistakes in front of customers.
- **Workflow automation with agents** is the most ambitious category: the model doesn't just answer, it **takes actions** (calls tools, fills forms, sends emails).
- Many use cases are built with the same techniques covered later: **prompt engineering, RAG, finetuning and agents**.

Before building, ask

1. **Why build it?** Does it solve a real problem, or is it just to keep up?
2. **What role does AI play?** Is it critical to the product or a nice extra? Does it act on its own (proactive) or only respond (reactive)?
3. **Is a human in the loop?** Does AI make the final decision, or does it help a person decide?

Planning AI applications

Building a demo with a foundation model is easy. Building a product people rely on is hard. This section is about planning before you build.

1. Use case evaluation: why build this?

The book ranks the reasons from most to least urgent:

1. **Existential threat**: if you don't use AI, competitors who do could make you obsolete (e.g. businesses built on information processing, creative work or customer support).
2. **Missed opportunity**: you could boost productivity or profit (better customer support, more leads, cheaper operations).
3. **Fear of missing out**: you don't know where AI fits yet, but you don't want to be left behind. It's fine to experiment, but be honest that this is the reason.

2. The role of AI in the product

Question	Option A	Option B
Critical or complementary?	Critical: the app doesn't work without AI (Face ID). Needs very high accuracy.	Complementary: AI is a nice extra (Smart Compose). Mistakes are tolerable.
Reactive or proactive?	Reactive: answers when the user asks (chatbot). Speed matters.	Proactive: acts without being asked (traffic alerts in Google Maps). Can take its time, but must be worth the interruption.

Question	Option A	Option B
Dynamic or static?	Dynamic: updates per user from their feedback (Face ID learning your face as it changes).	Static: one model for everyone, updated from time to time.

3. The role of humans: human-in-the-loop

How much does AI decide on its own? Microsoft's **Crawl-Walk-Run** framework increases automation step by step:

- **Crawl**: a human must be involved in every decision.
- **Walk**: AI can work directly with internal employees.
- **Run**: more automation, possibly AI talking directly to external users (customers).

4. Defensibility: can competitors copy you?

The low barrier to entry is a blessing and a curse: if it's easy for you to build, it's easy for others too. Your app may also become a feature that the model providers add themselves (e.g. "chat with your PDF").

Three kinds of competitive advantage:

- **Technology**: hard to win here, because everyone uses similar foundation models.
- **Distribution**: usually belongs to big companies with existing users.
- **Data**: the best bet for startups. Getting to market early collects **usage data**, which improves the product, which attracts more users (a **data flywheel**).

5. Setting expectations: define success

Decide how you'll measure success **before** building. For a customer support chatbot:

- **Business metrics**: % of messages handled automatically, how many messages it can process, how fast it responds, how much human work it saves.
- **Usefulness threshold**: the minimum quality before it's worth showing to users.

Other metrics to track:

Metric type	Examples
Quality	Are the responses correct and helpful?
Latency	**TTFT** (time to first token), **TPOT** (time per output token), total latency
Cost	Cost per request (inference)
Other	Interpretability, fairness

6. Milestone planning: the last mile is the hardest

- Start by testing existing models to see what's possible.
- **Early progress is misleading.** Going from 0 to 60 is quick; going from 60 to 100 is slow.
- Example from the book: LinkedIn reached about **80%** of the experience they wanted in **1 month**, then needed **4 more months** to get past **95%**, mostly fixing mistakes and hallucinations.

7. Maintenance: the ground keeps moving

- New models come out often and get **better and cheaper**, so plan to re-evaluate and possibly switch models. Keep your app easy to change.
- Something you build yourself today may become a cheap off-the-shelf feature tomorrow. Weigh that before investing heavily.
- Outside factors can change too: **regulations**, intellectual property rules, and access to compute (GPUs).

Summary: Before building: know **why** (use case), decide **what role** AI and humans play, think about **defensibility**, set **success metrics**, plan for the **last mile**, and expect **constant change**.

The AI engineering stack

Every AI application, whatever the use case, is built on the same **three layers**. Most AI engineers start at the top and only go deeper when they need to.

Layer	What happens here	Examples
1. Application development	Give the model good instructions and context, build the interface, evaluate the results	Prompts, RAG, chat UI, evaluation
2. Model development	Change or improve the model itself	Training, finetuning, datasets, inference optimisation
3. Infrastructure	Run the models and keep them healthy	Serving, GPUs/compute, monitoring

Many principles stay the same as in ML engineering: you still need to define the business problem, set metrics, and evaluate the system. What changes is the scale of the models and how much of the work is adaptation and evaluation.

AI engineering vs ML engineering

1. **No training from scratch**: you start with a model someone else trained, so the work is **adapting** it rather than building it.
2. **Bigger models**: more compute and higher latency, so **inference optimisation** matters more.
3. **Open-ended outputs**: there is no single right answer, so **evaluation** is much harder.

In short: AI engineering is less about developing models and more about **adapting and evaluating** them.

Two ways to adapt a model

	Prompt-based (prompt engineering, RAG)	Finetuning
Changes model weights?	No	Yes
Data needed	Little or none	More
Effort	Easy, quick to try	More complex
Benefit	Fast to get started	Can improve quality, latency and cost

Start with prompts; move to finetuning only if prompts aren't enough.

Layer 2: model development (three parts)

1. **Modeling and training**
   - **Pre-training**: training a model from scratch (the most expensive step, usually done by big labs).
   - **Finetuning**: continuing to train an already-trained model on your own data.
   - **Post-training**: the same idea as finetuning, but usually done by the model developer (e.g. to make the model follow instructions) before release.
2. **Dataset engineering**: collecting, cleaning, labelling and generating data. Foundation models use mostly **unstructured** data (text, images), so the work is things like removing duplicates, tokenising, filtering toxic or low-quality content, and checking data quality.
3. **Inference optimisation**: making models **faster and cheaper** to run, e.g. **quantization** (smaller numbers per parameter), **distillation** (training a small model to copy a big one), and parallelism.

Layer 1: application development (three parts)

1. **Evaluation**: checking that the app works and reducing risk. It's needed at every step (choosing a model, comparing prompts, monitoring in production) and is harder because outputs are open-ended.

2. **Prompt engineering and context construction**: getting the behaviour you want from the **input alone**, without changing weights. Context construction means giving the model the information, memory and tools it needs (e.g. RAG).
3. **AI interface**: how users reach the app: standalone web/mobile apps, browser extensions, chatbots inside Slack or WhatsApp, plugins in tools like VS Code. Good interfaces also **collect user feedback**, which improves the app.

Application work	ML engineering	AI engineering
AI interface	Less important	Important
Prompt engineering	Not applicable	Important
Evaluation	Important	**More important**

AI engineering vs full-stack engineering

Because so much of the work is now in the application layer (interfaces, APIs, feedback), AI engineering looks a lot like **full-stack development**. Many tools now support JavaScript, not just Python (LangChain.js, Transformers.js, the Vercel AI SDK), so web developers can build AI apps too.

The order of work also flips:

- **Traditional ML:** Data → Model → Product. You collect data and train a model before anyone can try the product.
- **AI engineering:** Product → Data → Model. You build a product on an existing model first, and invest in data and your own models only if the product shows promise. This means faster feedback from real users.

Summary: Three layers (application, model, infrastructure). AI engineering spends most of its time on the application layer, **adapting** and **evaluating** models, and builds the **product first**.

Chapter 1 summary

The big idea: Language model → (self-supervision + scale) → Large Language Model → (more modalities + general purpose) → Foundation Model → (adapted to real problems) → **AI Engineering**.

Key takeaways

1. **Foundation models** are large, general-purpose models trained with self-supervision on huge amounts of data. One model can handle many tasks and many types of data.

2. **AI engineering** builds applications on top of these models. The barrier to entry is low (an API key is enough to start), which is why the field grew so fast.
3. **Use cases** fall into 8 categories: coding, image & video, writing, education, conversational bots, information aggregation, data organization and workflow automation.
4. **Planning matters more than the demo**: know why you're building, decide the roles of AI and humans, think about defensibility, set metrics, expect the last mile to be slow, and plan for constant change.
5. **The stack** has three layers. Most AI engineering happens in the application layer: **adapting** models (prompts, RAG, finetuning) and **evaluating** them.

The process of building an AI system

Like the software development lifecycle (SDLC), building an AI application is a **loop**, not a straight line. Evaluation sits in the middle because it happens at every stage.



Phase	Stage	Key question	Covered in
Plan	1. Use case	Why build this? Is it a threat, an opportunity or just FOMO?	Planning AI applications
	2. Roles & metrics	Is AI critical or complementary? Is a human in the loop? What does success look like	Planning AI applications

Phase	Stage	Key question	Covered in
		(quality, latency, cost)?	
Build	3. Pick a model	Which existing model gets closest? API or open source?	AI engineering stack
	4. Adapt the model	Can prompts do it? Do I need RAG? Is finetuning worth it?	AI engineering stack
	5. Build the interface	Where will users reach it (app, chat, plugin)? How do I collect feedback?	AI engineering stack
Run	6. Deploy gradually	Crawl → Walk → Run: start with humans checking, then automate more	Planning AI applications
	7. Monitor & learn	Is it meeting the metrics? Is usage data improving it (data flywheel)?	Planning AI applications
	8. Maintain	Is there a better or cheaper model? Have costs or rules changed? → back to step 1	Planning AI applications
Throughout	Evaluate	Is it good enough yet? Open-ended outputs make this the hardest part	Whole chapter

How this differs from the traditional SDLC:

- **Product first:** you ship on an existing model and only invest in data and training once the product proves itself (Product → Data → Model).
- **Evaluation never stops:** outputs are open-ended, so "testing" is ongoing, not a single phase before release.
- **The ground moves:** new models arrive every few months, so the loop goes round more often than in normal software.